

Name: K.Likitha

Hall Ticket: 2303A52144

Batch: 41

Task 1: Problem Identification & AI-Assisted Ideation

Prompt:

Build a Civic Complaint Management System for a smart city where users can submit complaints with location tagging, track status in real time, receive notifications, and allow admins to manage, assign, and monitor complaints through a dashboard. Ensure the system improves transparency, reduces delays, and supports citizens, municipal staff, and field workers.

Code:

```
Lab23_tests.py U X
Lab23_tests.py > ...
1  """Unit tests for Lab 23 capstone complaint management system."""
2
3  import os
4  import tempfile
5  import unittest
6
7  from Lab23_capstone.service import ComplaintService
8  from Lab23_capstone.storage import JSONStorage
9
10
11 class TestComplaintService(unittest.TestCase):
12     """Validate complaint service behavior and edge cases."""
13
14     def setUp(self) -> None:
15         self.temp_dir = tempfile.TemporaryDirectory()
16         self.data_file = os.path.join(self.temp_dir.name, "lab23_test_data.json")
17         self.service = ComplaintService(JSONStorage(self.data_file))
18
19     def tearDown(self) -> None:
20         self.temp_dir.cleanup()
21
22     def test_create_complaint_success(self) -> None:
23         complaint = self.service.create_complaint(
24             citizen_name="Ananya Rao",
25             area="Ward 5",
26             issue_type="Garbage Overflow",
27             description="Bin has not been cleared for 4 days.",
28             priority="High",
```

```

class TestComplaintService(unittest.TestCase):
    def test_persistence_across_service_reloads(self) -> None:

        self.assertEqual(len(complaints), 1)
        self.assertEqual(complaints[0]["citizen_name"], "Rekha")

    def test_dashboard_metrics_counts(self) -> None:
        self.service.create_complaint(
            citizen_name="Ravi",
            area="Ward 1",
            issue_type="Garbage",
            description="Garbage pickup delayed.",
            priority="Critical",
        )
        self.service.create_complaint(
            citizen_name="Pooja",
            area="Ward 1",
            issue_type="Drainage",
            description="Drain blockage on main road.",
            priority="High",
        )

        self.service.update_status(1, "In Progress")
        self.service.update_status(2, "Resolved")

        metrics = self.service.dashboard_metrics()

```

Output:

```

Smart City Complaint and Waste Tracking Platform
1. Register new complaint
2. View all complaints
3. Filter complaints
4. Update complaint status
5. Assign staff
6. Dashboard metrics
7. Exit
Select an option: 1
Citizen Name: virat
Area/Ward: warangal
Issue Type (e.g., Garbage, Pothole): garbage
Description: not good
Select priority:
1. Low
2. Medium
3. High
4. Critical
Select option: 3
Complaint created successfully.
[#1] garbage | Area: warangal | Priority: High | Status: Open
Citizen: virat | Assigned: Unassigned | Created: 2026-04-12 13:37:10 | Updated: 2026-04-12 13:37:10
Description: not good
-----

```

Explanation:

I chose the Civic Issue Reporting System to solve the gap in reporting and tracking civic issues. The current manual process causes delays and lack of transparency. A digital system enables easy tracking, faster resolution, and better efficiency.

Task 2: System Planning & Architecture Design

Prompt:

Act as a system architect and design a simple prototype architecture for a smart city complaint system called CivicFix. Include frontend, backend, and database components, explain the data flow, and propose a basic schema for users and complaints. Also define the main APIs/modules required.

Code:

Architecture & Schema (Text-based):

- Frontend: CLI (Command Line Interface) for prototype (can be extended to React.js web app later)
- Backend: Python application using modular functions (input handling, complaint management, status update)
- Database: In-memory Python lists/dictionaries (can be upgraded to SQLite for real use)•

Data Flow:

User enters complaint → Backend validates input → System generates Ticket ID → Data stored in dictionary → Confirmation shown to user

- Database Schema:
 - Users: {user_id: INT, name: STR, role: STR [citizen, admin]}
 - Complaints: {ticket_id: INT, citizen_name: STR, category: STR, description: STR, status: STR [Open, In-Progress, Resolved]}
- APIs / Modules:
 - register_complaint()
 - view_complaints()
 - update_status()
 - get_complaint_by_id()

Explanation:

I chose a simple and modular architecture for quick development and easy testing. The CLI-based frontend and Python backend keep the system clear and manageable, while in-memory storage reduces complexity for the prototype. This design is flexible and allows future upgrades like adding a database or web interface without changing core logic.

Task 3: Modular Code Development with AI

Prompt:

Write a modular Python program for a CivicFix complaint system with functions for registering complaints, viewing and updating status, and storing data using a list of dictionaries. Include proper input handling and error handling to keep the structure clean and manageable.

Code:

```
# CivicFix Complaint Management System

complaints_db = []
ticket_counter = 1

def register_complaint(name, category, description):
    global ticket_counter

    try:
        name = name.strip()
        category = category.strip()
        description = description.strip()

        if not name or not category or not description:
            return "Error: All fields are required."

        complaint = {
            "ticket_id": ticket_counter,
            "citizen_name": name,
            "category": category,
            "description": description,
            "status": "Open"
        }
        complaints_db.append(complaint)
        ticket_counter += 1
```

```

        complaints_db.append(complaint)
        ticket_counter += 1

        return f"Success! Complaint registered with Ticket ID: {complaint['ticket_id']}"

    except Exception as e:
        return f"Error while registering complaint: {e}"

def view_complaints():
    try:
        if not complaints_db:
            return "No complaints found."

        output = "\n--- All Complaints ---\n"
        for complaint in complaints_db:
            output += (
                f"ID: {complaint['ticket_id']} | "
                f"Name: {complaint['citizen_name']} | "
                f"Category: {complaint['category']} | "
                f"Status: {complaint['status']}\n"
            )
        return output

```

```

    except Exception as e:
        return f"Error while viewing complaints: {e}"

def update_status(ticket_id, new_status):
    valid_statuses = ["Open", "In-Progress", "Resolved"]

    try:
        ticket_id = int(ticket_id)

        if new_status not in valid_statuses:
            return "Error: Invalid status."

        for complaint in complaints_db:
            if complaint["ticket_id"] == ticket_id:
                complaint["status"] = new_status
                return f"Ticket {ticket_id} updated to {new_status}."

        return "Error: Ticket ID not found."

    except ValueError:
        return "Error: Ticket ID must be an integer."
    except Exception as e:
        return f"Error while updating status: {e}"

```

```

        return f"Error while updating status: {e}"

def main():
    while True:
        print("\n--- CivicFix Complaint System ---")
        print("1. Register Complaint")
        print("2. View Complaints")
        print("3. Update Complaint Status")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            name = input("Enter citizen name: ")
            category = input("Enter complaint category: ")
            description = input("Enter complaint description: ")
            print(register_complaint(name, category, description))

        elif choice == "2":
            print(view_complaints())

        elif choice == "3":
            ticket_id = input("Enter Ticket ID: ")
            new_status = input("Enter new status (Open/In-Progress/Resolved): ")
            print(update_status(ticket_id, new_status))

        elif choice == "4":
            print("Exiting CivicFix System.")
            break

        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()

```

Output:


```
--- CivicFix Complaint System ---
1. Register Complaint
2. View Complaints
3. Update Complaint Status
4. Exit
Enter your choice: 1
Enter citizen name: naziya
Enter complaint category: road
Enter complaint description: open
Success! Complaint registered with Ticket ID: 1

--- CivicFix Complaint System ---
1. Register Complaint
2. View Complaints
3. Update Complaint Status
4. Exit
Enter your choice: 4
Exiting CivicFix System.
PS C:\Users\sdnaz\OneDrive\Desktop\ai assiant coding> |
```

Explanation:

I used AI to create modular code with separate functions for registering, viewing, and updating complaints. This improves clarity and maintainability, while error handling prevents crashes. Using lists and dictionaries keeps the design simple and suitable for a prototype.

Task 4: AI-Assisted Testing, Debugging & Optimization

Prompt:

Review the CivicFix Python code, generate test cases, identify potential bugs, and provide an optimized version with better input validation, proper try-except error handling, and a cleaner code structure.

Code:

Test Cases & Bug Identification:

1. **Test Case 1 (Valid Input):** Register complaint with valid strings. *Result: Pass.*
2. **Test Case 2 (Invalid Ticket ID):** Try to update status of Ticket ID 99 or passing a string instead of an int. *Result: Bug identified - code fails silently or crashes if a string is passed instead of an integer ID.*
3. **Test Case 3 (Empty Input):** Pass empty strings to register_complaint. *Result: Pass (handled by existing basic validation).*

Optimized Code:

```
complaints_db = [] ticket_counter =  
1
```

```
def register_complaint(name: str, category: str, description: str) -> str: global  
    ticket_counter
```

```
    try:
```

```
        # Optimization: Stripping whitespace
```

```
        name, category, description = name.strip(), category.strip(), description.strip() if not  
        name or not category or not description:
```

```
            raise ValueError("Fields cannot be empty or just spaces.")
```

```
        complaint = {
```

```
            "id": ticket_counter,
```

```
            "name": name, "category":
```

```
            category, "description":
```

```
            description, "status":
```

```
            "Open"
```

```
        }
```

```
        complaints_db.append(complaint)
```

```
        ticket_counter += 1
```

```
        return f"Success: Ticket ID {complaint['id']} created." except  
    Exception as e:
```

```
        return f"Error registering complaint: {e}"
```

```
def update_status(ticket_id: int, new_status: str) -> str:
```

```
    valid_statuses = ["Open", "In-Progress", "Resolved"] try:
```

```
        ticket_id = int(ticket_id) # Type enforcement optimization if
```

```
        new_status not in valid_statuses:
```

```
            raise ValueError("Invalid status provided.")
```

```
    # Optimization: using next() with a generator expression for faster lookup O(N) complaint =  
    next((c for c in complaints_db if c["id"] == ticket_id), None)
```

```
    if complaint:
```

```
        complaint["status"] = new_status
```

```
        return f"Success: Ticket {ticket_id} updated." else:
```


Output:

(Testing Optimized Code)

Output for `update_status("abc", "Resolved")`: Input Error: invalid literal for int() with base 10: 'abc'

Explanation:

The initial code was weak in handling data types, causing crashes on invalid input like "one" instead of 1. I improved it using input sanitization with `.strip()`, proper try-except blocks, and clearer type handling. I also optimized search using a generator expression (`next(...)`) to stop once the ID is found.

Task 5: Documentation, Version Control & Reflection**Prompt:**

Generate a concise README.md for the CivicFix app covering setup and usage, suggest 5 standard Git commit messages, and include a short reflection on AI usage, limitations, and future improvements.

README Snippet:

CivicFix: Smart City Complaint System

A lightweight CLI application for reporting and tracking civic issues.

Setup

1. Ensure Python 3.8+ is installed.
2. Clone the repository: ``git clone https://github.com/...``
3. Run the application: ``python app.py``

Usage

- Developers can import ``register_complaint`` and ``update_status`` from ``app.py``.
- Data is stored in-memory per session.

Simulated Git Commit Messages:

1. init: setup initial project structure and README
2. feat: add `register_complaint` function and core data dictionary
3. feat: implement ticket status update logic
4. fix: add input sanitation and try-except error handling
5. docs: add python docstrings to core logic functions

Explanation:

Proper documentation and version control distinguish basic scripts from professional software. Using AI to generate the README and commit history helped me learn standard practices. It also showed that while AI is useful, the developer must guide the design to ensure good software engineering principles.